

An AgentOS Needs a Formal Representation of Agents

Qin Liu, Boyuan Shao, Xinhao Wang, Yuyang Qiu, and Fengshan Zhao*

State Key Laboratory of Novel Software Technology, Nanjing University

Nanjing, China

fs.zhao@nju.edu.cn

Abstract

Everyone is building an “AgentOS.” Three camps currently claim the term: schedulers that treat the LLM as a CPU, memory managers that page the context window, and infrastructure stacks that bundle orchestration, tools, and observability. All three contribute real machinery, but none gives the runtime a formal contract for what an agent *is*, what it may *touch*, or what resource envelope it needs before it runs. We argue that AgentOS needs a formal representation of agents: typed λ -terms for executable structure, inferred effects for capability boundaries, and cost grades whose enforcement conditions are explicit. We support the position with a running prototype—an 11-construct agent calculus executed by a CEK machine, validated against 835 real-world agent configurations—and close with the open problems this stance exposes, from cost soundness under LLM uncertainty to cross-agent fair scheduling.

1 Introduction

Every agent framework now ships with an operating system in its name. The trade press calls 2025–26 the year of the “AgentOS”; research prototypes and commercial stacks alike advertise schedulers, memory managers, and tool managers for fleets of LLM agents. But an operating system is not one mechanism; it is a layered chain of responsibilities. In an AgentOS, humans own goals, checklists, and exit conditions; clients mediate interaction with a project; agent programs organize work into executable structure; runtimes execute, trace, schedule, and roll back; infrastructure provides resources, isolation, CI, and security boundaries (Figure 1). The question is what crosses the central boundary: what does the runtime know about the agent program it is asked to manage?

Three camps currently define the term, and each has earned its analogy before it reaches its limit. The **LLM-as-kernel** camp, exemplified by AIOS [1], makes LLM inference a shared scarce resource and imports queues, priorities, and schedulers for multiplexing agents over it. But its scheduling decisions are driven by *post-hoc measurement*: the system learns what an agent costs only by running it. A scheduler that discovers a job’s resource demand after the job finishes is not scheduling; it is billing. The **virtual-memory** camp, exemplified by MemGPT and its successor Letta [2], names the context window as the agent’s working set and gives long-running agents a memory hierarchy. But the pager has no semantic visibility into the agent program it serves: it cannot know which effects must be retained, which intermediate state is dead, or which memory the program will touch next—the information that every real virtual-memory system extracts from program structure. The **infrastructure** camp solves the practical glue problem by bundling orchestration, tool registries, memory, and observability. Without a definition of what a process *is*, however, its isolation,

protection, and metering are promises of the deployment diagram, not properties of the system. In all three cases, the contribution is real machinery; the missing piece is the contract between the agent program layer and the runtime/control plane.

The contract at this boundary must answer three OS-style questions: **what runs** (the process abstraction), **what it can touch** (protection), and **what it costs** (accounting and scheduling). Our position is that for agents these three answers should be *formal*, *compositional*, and *inferable before execution*. Concretely: an agent should be a typed term in a small calculus; its authority should be an effect, inferred from its structure rather than configured beside it; and its resource demand should be a graded cost bound, computed at compile time and consumed by the scheduler as a first-class primitive. We use *should* deliberately—this paper takes a normative stance on what a formal AgentOS representation should provide, and offers a prototype only as evidence of feasibility, not as a product.

The prototype exists and runs today. It implements a λ -calculus of agents with eleven constructs, in which YAML agent configurations are serialized λ -terms and execution is β -reduction on a CEK machine [3]; an effect algebra over pure, state, io, and llm with serial, parallel, and iterated composition; and a graded cost semantics that assigns every term a pre-execution cost envelope over tokens, latency, and money. Token and dollar components are enforceable only with bounded inputs and runtime checks; latency is deployment conditional. The prototype core is ~10K lines of Python, passes 175 core tests, and its static pipeline has been exercised on 2,225 agent configurations scraped from open-source repositories, of which 881 parse and 835 pass a lint of 26 rules. On a 29-run benchmark across 12 pipeline shapes, every static token reserve contained the provider-reported consumption (§5).

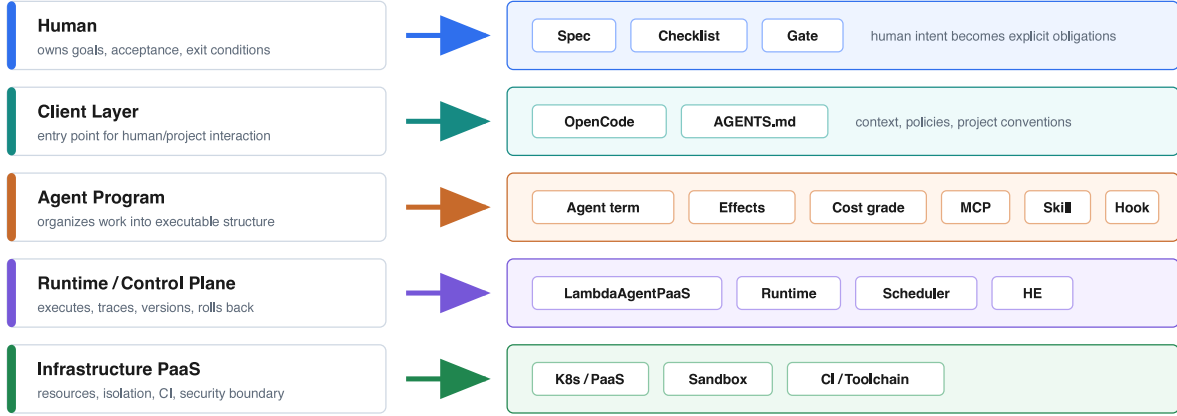
This paper makes four contributions. (1) We argue that current AgentOS designs, whatever their engineering merits, lack a contract across the agent program/runtime boundary—and that this is the field’s central gap (§2). (2) We give a concrete formal representation: typed agent terms for *what runs*, an effect algebra for *what it can touch*, and graded cost envelopes for *what it costs* (§3). (3) We show how that representation is consumed by the rest of an AgentOS: admission control and deadline scheduling over stated envelopes, capabilities by inference rather than configuration, and semantics-preserving runtime optimization (§4). (4) We report the prototype’s status honestly—including which theorems remain paper proofs and which system layers remain unbuilt—and offer the gaps as open problems for the community (§5).

2 The Missing Contract

Figure 1 is not a product diagram; it is a responsibility diagram. The Human layer decides what should be done and when it is acceptable to stop. The Client layer supplies project context and

*Fengshan Zhao is the corresponding author.

AgentOS as layered responsibilities



Top-down: people set obligations; clients provide context; agent programs expose structure; runtime schedules and audits; infrastructure enforces isolation. Formal representation is the Agent Program / Runtime contract: typed term + effects + cost grade.

Figure 1: AgentOS as layered responsibilities. The formal representation is the contract between the agent program and runtime layers.

interaction conventions. The Agent Program layer turns a task into executable structure. The Runtime/Control Plane executes that structure, tracks it, schedules it, and rolls it back. The Infrastructure layer enforces resource and security boundaries. The gap sits between the middle two layers: the runtime is asked to manage an agent program, but current systems usually hand it an opaque prompt, a tool list, and telemetry after the fact.

This is exactly the place where operating systems historically required a core OS abstraction. Strip an operating system to its core and three responsibilities remain. It defines the *process*: the unit that runs, with enough structure that the system can reason about it. It enforces *protection*: a process touches the world only through a syscall interface, and rings bound what each layer may do. And it performs *accounting and scheduling*: resources are metered in units the runtime understands—time slices, pages—and allocated *before* use, not merely tallied after. Unix earned its longevity because these three answers were crisp enough to build everything else on.

Figure 2 lines up the three responsibilities against today’s AgentOS landscape and against the formal representation we advocate. The contrast is starkest in the middle column. Asked *what runs*, current systems answer: a prompt plus a tool list. Asked *what it can touch*, they answer: whatever the tool allowlist says, maintained by hand beside the agent rather than derived from it. Asked *what it costs*, they answer: we will tell you when it finishes. Each answer is operationally serviceable and semantically empty.

The cost answer deserves the closest scrutiny, because scheduling is where the emptiness bites. The point is simple: a scheduler cannot wait until a job is over to learn whether it should have admitted that job. Admission control must decide, before execution, whether a new job can be accepted without breaking existing commitments. Deadline scheduling must decide, before execution, whether the job can finish by time D . If the system only learns tokens, latency, and money after the agent finishes, it can bill, throttle, or kill future jobs, but it cannot promise anything about this one. Real-time operating systems made the same distinction decades ago:

since Liu and Layland’s rate-monotonic analysis [4], hard real-time schedulers have relied on a *worst-case execution time* (WCET), a static upper bound on how long each task can run. With that bound, the scheduler can check whether the task set fits. Without it, there is no schedulability analysis and no deadline guarantee. Agents do not already have classical WCET; the analogy is that schedulers need a pre-execution envelope with explicit enforcement conditions. That envelope is the missing contract primitive.

The standard objection is that LLMs make such bounds impossible: output length and prompt growth are random variables. The objection conflates *prediction* with *bounding*. Agent programs still expose structural ceilings: maximum-output settings bound generated tokens, `max_steps` caps loops, and `retry` caps guards. Total billed tokens additionally need input-size contracts and a runtime check. Uncertainty affects how *tight* the bound is, not whether the properly scoped claim holds.

3 A Formal Agent Representation, Concretely

This section gives the three representation primitives in just enough formality to show they are real, each with the same worked example and a pointer to its implementation. Throughout, consider a three-stage retrieval pipeline—an agent that looks up documents, drafts a JSON answer with validation and retry, then summarizes:

```
pipeline = retrieve
          >> Guard(draft, valid_json, retry=2) >> summarize
```

where *retrieve* is a tool (a pure lookup into a corpus), *draft* and *summarize* are LLM calls against a model M , and the *Guard* retries *draft* up to twice until its output parses as a JSON object.

3.1 What runs: typed agent terms

The first primitive replaces “a prompt plus a tool list” with a term in a small calculus. Every agent configuration—YAML, JSON, or programmatic—is a serialized λ -term; compiling it (`from_config`) is desugaring; running it is β -reduction. The calculus has eleven

OS responsibility	Classic OS answer	“AgentOS” today	Formal representation (this paper)
what runs (process)	process = address space + PC	“a prompt + a tool list” — no definition	typed agent term, 11 constructs $\tau_1 \rightarrow^\varepsilon \tau_2$ core.py
what it can touch (protection)	syscall interface + protection rings	ad-hoc tool allowlists — no formal guarantee	effect algebra = capability $\varepsilon ::= \text{pure} \mid \text{llm} \mid \text{io} \mid \text{state}$ $\varepsilon \cdot \varepsilon, \varepsilon / \varepsilon, \varepsilon^n$ effects.py · tool_gateway.py
what it costs (accounting & scheduling)	CPU quanta, memory pages (a priori)	post-hoc billing — measured after the run	graded cost bound, pre-execution $g = (p, p_fatal, t, l, m)$ cost_grade.py · cek_machine.py

Figure 2: Three OS responsibilities, and what answers them.

constructs: Lam (an LLM call—a λ -abstraction whose body is an oracle), application, Compose, If, bounded Loop, Pair with projections Fst/Snd, Tool, Route, Guard, and Memory. Each is ordinary λ -calculus under the hood—If is the Church conditional, Loop a fuel-bounded Y combinator, Pair a Church pair—which is precisely what makes the rest of the representation useful: structure that can be analyzed is structure that can be typed, bounded, and scheduled.

Terms are typed: an agent has a function type $\tau_1 \xrightarrow{\varepsilon} \tau_2$, where the arrow carries an *effect* ε (next section) and the base types include JSON-Schema-structured records with width/depth subtyping. Our running example types as $\text{Str} \xrightarrow{\varepsilon} \text{Str}$: it consumes a question and produces a sentence, whatever its internal shape.

The payoff is that “process” stops being a metaphor. A typed term has static structure the runtime and static checker can traverse—before execution—to infer everything in §3.2 and §3.3. The construct set is implemented in core.py; its static pipeline (26 lint rules over the term structure) has been validated on the 2,225-config corpus mentioned above, and the small-step semantics (23 reduction rules) runs on a CEK machine whose every transition is observable and metered.

3.2 What it can touch: effects as capabilities

The second primitive replaces the tool allowlist with an *effect algebra*:

$$\varepsilon ::= \text{pure} \mid \text{llm}(m) \mid \text{io} \mid \text{state}(s) \mid \varepsilon \cdot \varepsilon \mid \varepsilon \parallel \varepsilon \mid \varepsilon^n$$

ordered by a lattice $\text{pure} \leq \text{state} \leq \text{io} \leq \text{llm}$, with serial ($\cdot$), parallel ($\parallel$), and iterated (n) composition mirroring the three ways terms compose. Effects are *inferred* from the term—the inference pass walks the structure—never declared beside it. For the running example the inference yields

$$\text{io} \cdot \text{llm}(M) \cdot \text{llm}(M)$$

a read from the corpus, then two LLM stages against model M (the guard’s retry multiplicity surfaces in the cost grade, §3.3, rather than in the effect).

Read the inferred effect as a *capability envelope*. The prototype infers coarse classes—pure, state(s), io, and llm(M)—while a production AgentOS should refine io into resource-indexed effects such as fs.read(p), fs.write(p), net(d), and secret(a). Enforcement

today lives in the tool gateway: allowlists, path/host ACLs, risk classification, and human confirmation. For example, a coarse effect $\text{io} \cdot \text{llm}(M)$ could be refined in production to $\text{fs.read}(\text{corpus}://\text{docs}/*)$ and $\text{llm}(M, \text{max-output}=600)$, while denying fs.write , $\text{secret}(*)$, and network destinations outside the declared provider, separating approved corpus reads from private-file exfiltration. Two properties make the envelope trustworthy under composition: composition never *removes* effects, and inference handles parallel branches and handoffs by adding each branch’s resources and those of any agent receiving control. Implementation: effects.py, tool_gateway.py.

3.3 What it costs: graded cost bounds

The third primitive is the one current AgentOS designs lack entirely: a *pre-execution* resource bound, computed from term structure, consumed by the scheduler. We grade each term with a static envelope

$$g = (p, p_fatal, t, l, m)$$

—success probability, per-step fatal-failure rate, and envelopes for tokens, latency (seconds), and money (USD). Grades compose by four rules that mirror the term constructors (Table 1).

A grade is a contract boundary, not magic prediction. We use three enforcement levels: *structural* facts from output caps, per-call reserves, loop fuel, retries, and branches; *runtime-enforced* token/dollar ceilings, which require bounded inputs, pre-call tokenization, and post-call settlement; and *deployment-conditional* latency, which needs bounded queues, tool timeouts, known serving configuration, and real parallel execution. Without those premises, the scalar token number is a reserve or estimate, not a guaranteed total-token upper bound. The prototype implements structural grading, post-run metering, and budget-exception hooks; residual enforcement is future work.

Today the tuple is coarse aggregate: each LLM term gets a model-level token reserve, and validation compares composed reserves with provider-reported total_tokens. A serving grade should refine tokens to $(t_{in}, t_{cache}, t_{out})$, price input, cached input, and output tokens separately, and split latency into queueing, prefill, and decode: $q + \ell_{prefill}(t_{in} - t_{cache}) + \ell_{decode}(t_{out})$. The AgentPaaS run path already meters input, output, cache-read, and cache-creation tokens; with prompt/tool/memory size contracts, the same algebra can fold them into static grades.

Table 1: Grade composition mirrors term composition (t, l, m components).

Construct	Rule	Reading
Compose	$t_1+t_2, l_1+l_2, m_1+m_2$	a pipeline’s budget is the sum
Pair/Par	$t_1+t_2, \max(l_1, l_2), m_1+m_2$	buys latency, never tokens
Loop ($\leq n$)	nt, nl, nm	bounded iteration, bounded cost
Guard (k retries)	$(1+k)t, (1+k)l, (1+k)m$	the retry budget is explicit

Tool latency follows the same rule: local tools are hard-bounded only inside a sandbox with CPU, memory, and I/O limits; remote tools are conditionally bounded by timeout, retry, and service-level assumptions; unbounded external services remain estimates unless wrapped by an enforceable timeout or killable sandbox boundary.

For parallel composition, latency is $\max(l_1, l_2)$ only when the runtime actually runs independent branches in parallel; if this prototype executes them serially, observed latency is closer to $l_1 + l_2$.

For the running example with $M = \text{qwen3-max}$, the system derives, statically,

$$t_{\text{res}} \leq 2,400 \text{ reserved tok}, \quad l \leq 4.1 \text{ s}, \quad m \leq \$0.0024, \quad p \geq 97\%$$

(the tool stage contributes latency only; the guarded draft contributes $3\times$ its single-call grade; the summarizer one more). This is four qwen3-max per-call reserves, not a prompt-growth cap. One run used 168 provider `total_tokens`; all 29 runs fit the reserve (Figure 4). The low 4–15% reserve utilization is a soundness–tightness tradeoff. Tightness can improve through input-size types, branch-sensitive grades, profile-guided constants, and residual grading.

Two runtime mechanisms close part of the loop. The CEK machine records each transition in a monotone cost vector and can raise a budget exception carrying the current state; in the present CEK path, LLM token and money accounting is not yet connected to provider usage, so this is an enforcement hook rather than complete cost containment. Separately, the AgentPaaS run path records provider-reported usage after execution, and a validation hook can compare finished runs with static grades. Figure 3 assembles the full contract: typed term, effect envelope, cost grade, scheduler decision, and metered runtime. Implementation: `cost_grade.py`, `cek_machine.py`.

The over-budget semantics is explicit. Before an LLM or tool call, a complete runtime tokenizes the prompt plus output cap and rejects, pauses, or requests a larger envelope if the residual grade no longer fits. After a call, provider-reported overflow is a contract violation; policy can abort, rollback, or require manual recovery. The prototype implements only pieces of this path.

The same machinery suggests a dynamic scheduler. Static admission consumes the whole grade; after execution starts, the CEK state $\langle C, E, K, \sigma, c \rangle$ exposes control, continuation, and spent cost. A residual estimator could reserve $c + g_{\text{rem}}(C, K)$, releasing unchosen branches, unused retries, and early loop fuel. The CEK state exposes the information this needs: after each transition, the scheduler can recompute the grade of the remaining control and continuation. The spent component c is fixed; the remaining envelope can shrink as branches are chosen, retries go unused, or loops exit early. Residual-grade recomputation and in-flight provider settlement are future work.

3.4 The contract, assembled

Each primitive answers one boundary question, but the design works because they are *the same structure read three ways*: the term is the process; the term’s effect is its capability; the term’s grade is its budget. Everything the scheduler needs is available before the first token is generated—what will run, what it may touch, what it can cost—and everything the runtime observes either confirms the static story or exposes a bug in the representation’s accounting, not in the agent. That closed loop, we argue next, is what the word “OS” should be reserved for.

4 What This Buys an AgentOS

A formal representation is not formality for its own sake. Its first job is to make the layers in Figure 1 talk through a shared object: the Human layer can attach acceptance gates to it; the Client layer can serialize it from project conventions; the Agent Program layer can compose it; the Runtime/Control Plane can schedule and audit it; and the Infrastructure layer can enforce the resources and isolation it implies. Each primitive of §3, read as an OS mechanism, enables something the current camps cannot build.

Scheduling on bounds, not bills. With a cost grade attached to every admissible term, *admission control* admits a task iff its envelope plus in-flight commitments fits capacity. *Deadline scheduling* is meaningful only when the latency grade has a matching serving contract: bounded queues, bounded tool calls, and a parallel executor for parallel terms. *Multi-tenant quotas* become enforceable ceilings when token and dollar budgets are checked before calls and metered after calls. The current PaaS service uses a coarse token reservation and post-run usage settlement; grade-based residual reservation is the systems work that turns the envelope into a hard ceiling. Telemetry-only schedulers can react after overspend; bounds make the commitment explicit.

Protection by inference, not configuration. The effect envelope gives an AgentOS its protection rings. A pure term can run without audit, anywhere, even speculatively; a state term is confined to its declared store; io and llm cross the gateway, where the coarse inferred effect is paired with concrete resource policy. Because the envelope is inferred, it cannot drift out of sync with the program the way a hand-maintained allowlist does; because composition is monotone, embedding a third-party sub-agent into a pipeline cannot silently widen—or launder—its authority. The same algebra covers the multi-agent constructs, so when one agent hands off to another, the handoff crosses a typed, effect-bounded boundary: IPC with semantics, rather than messages between black boxes.

Optimization with proofs, not vibes. The calculus satisfies six algebraic laws—compose associativity and identity, pair symmetry, loop unfolding, guard idempotence, route exhaustiveness—and these are runtime-grade licenses to transform programs. A runtime may fuse adjacent stages, batch or reorder footprint-commuting branches, cache and replay pure prefixes, and prove observable semantics unchanged. For parallelism, the sound claim is narrower: deterministic local-state branches may run in either order or in parallel only when neither writes what the other reads or writes; external effects must be absent, isolated, or commutative.

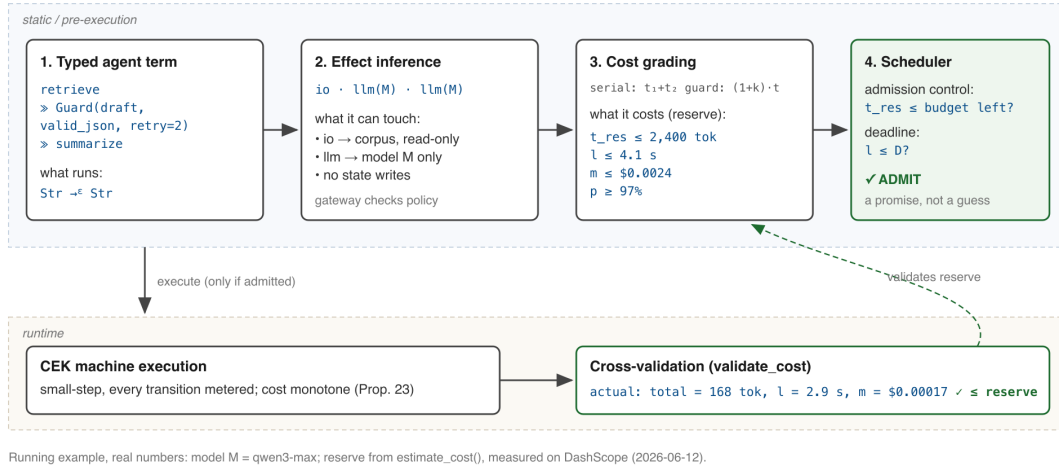


Figure 3: From agent term to scheduling decision before execution. Numbers are static envelopes; actuals are measured on qwen3-max.

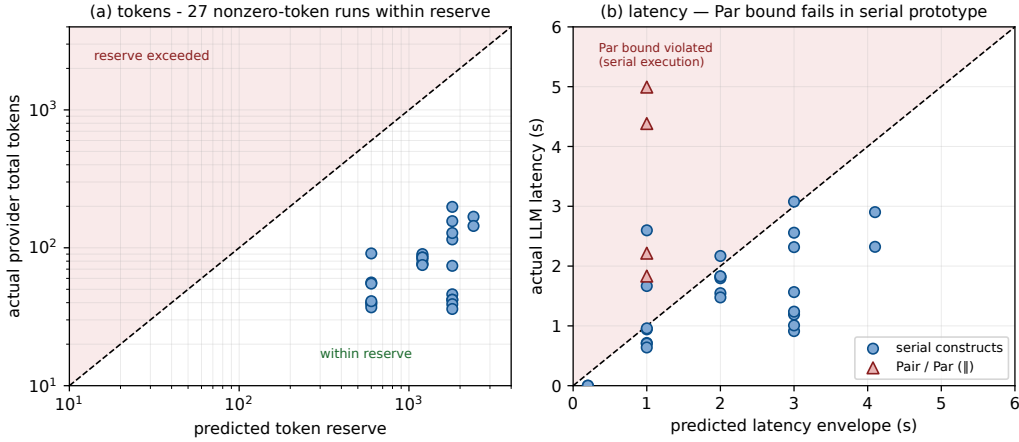


Figure 4: Predicted envelopes vs. actuals (29 runs, 12 pipelines, qwen3-max). (a) Static token reserves vs. provider total_tokens; all bounded prompts fit. (b) Red triangles are Pair/Par runs: the bound assumes parallel execution, but this CEK path serializes them, so measured latency exceeds the prediction.

These three are mutually reinforcing rather than merely adjacent. The effect system tells the scheduler which branches *may* run in parallel; the cost grade tells it whether parallelism is *worth it* (latency max versus token sum, Table 1); the algebraic laws guarantee the rewrite it chooses is *meaning-preserving*. The three boundary questions close into one loop over one structure—which is precisely what the AgentOS layers need, and what a bundle of orchestration services cannot offer by accretion.

5 Status, Limitations, and Open Problems

What runs today. The prototype core is ~10K lines of Python: the 11-construct calculus, effect inference, cost grading, and a CEK

machine implementing the 23-rule small-step semantics with per-transition metering. It passes 175 core tests. The static pipeline has been exercised against 2,225 real-world agent configurations (881 parse; 835 pass the 26-rule lint). Algebraic effect handlers decouple semantics from execution strategy—the same term runs under production, deterministic-test, and tracing handlers, with a (paper-proved) type-preservation theorem. And the central claim has empirical backing: across 29 runs of 12 pipeline shapes against a live model, every static token reserve contained provider-reported consumption—with measured usage at 4–15% of the reserve (Figure 4).

What we are not claiming. The metatheory—type safety, bounded termination, cost monotonicity, cost soundness—exists as careful

paper proofs, not mechanized ones; cost soundness in particular still lacks a semantic model of the LLM as a stochastic oracle, which is why we state it as a theorem schema validated empirically, and are mechanizing it as separate work. The systems layer is honest about being unbuilt: no multiplexing LLM-serving scheduler, no multi-tenant hardening, no container-grade isolation. The implemented cost model is also intentionally scalar: static t is a coarse per-call reserve, while runtime measurements record provider `total_tokens`. It does not yet derive prompt/tool/memory size contracts, separately price fresh input, cached input, and output tokens, or model prefill/decode latency. Runtime enforcement is also partial: token-budget and CEK budget-exception hooks exist, but the main Agent-PaaS path mostly records provider usage after execution, and the CEK LLM path has not yet wired provider token/money usage into its cost vector. Pre-call residual token and dollar checks remain future work. Parallel safety checks are partial too: they cover annotated-state write/write conflicts, not full read/write or external-effect noninterference. Figure 4 is the sharpest self-critique: the parallel latency bound $\max(l_1, l_2)$ was exceeded in 4 of 4 runs because the CEK evaluation path used in the experiment serializes Par. That bound specifies a parallel scheduler, not this execution path.

Open problems. We offer the gaps as invitations. (1) *Cost soundness under LLM uncertainty*: should bounds be worst-case or probabilistic, and what oracle model proves either? (2) *Cross-agent fairness*: what is fair-share scheduling over cost *bounds* rather than usage histories? (3) *Effect-directed context management*: can pagers use live state effects to evict memory without losing semantics? (4) *Tight and dynamic grades*: can profile-guided and residual grading tighten loose bounds while preserving admission control? Provider “reasoning budgets” are best viewed as primitive parameters of one model invocation, like output caps or reasoning effort. An AgentOS grade is the agent-level composition of many such invocations plus routes, loops, guards, retries, tools, memory growth, cache behavior, parallelism, and serving-aware latency.

6 Related Work

AgentOS systems. AIOS [1] multiplexes agents over LLM inference with OS-style schedulers and managers; MemGPT/Letta [2] virtualizes the context window. We regard both as the strongest versions of the systems-first approach—and as evidence for our thesis, since each builds mechanism that formal semantics would let it build *better*: AIOS’s scheduler lacks a pre-execution bound to schedule against; MemGPT’s pager lacks effect visibility to page with. Commercial “AgentOS” stacks bundle orchestration, tools, memory, and observability; they are not research artifacts and we engage them only as definitional pressure on the term.

PL foundations. Our formal toolkit is openly inherited. Graded and quantitative type systems (Granule [5]; coeffects [6]) track resource grades through typing; automatic amortized resource analysis [7] derives static cost bounds for functional programs; algebraic effects and handlers [8] separate effectful syntax from interpretation, which our handler layer implements directly; session types [9] type communication, as our effect algebra does for inter-agent constructs; the executable semantics is a CEK machine [3].

The delta. What is new is not the machinery but its object and its consumer. First, the resource being graded originates in an *LLM oracle call*: stochastic in size, priced by an external market, irreducible to static program text—none of the classical cost-analysis settings (AARA’s data-structure potentials, Granule’s usage counts) model it, and open problem (1) of §5 is exactly the gap. Second, the bound’s consumer is not a type checker but an *operating system*: admission control, deadlines, and quotas consume the grade as a scheduling primitive, the role WCET plays in real-time systems [4]. Graded types gave us the vocabulary; the position of this paper is that the AgentOS runtime/control plane is the place that vocabulary has been waiting to be spoken.

7 Conclusion

AgentOS should not mean merely a scheduler, a memory trick, or an infrastructure bundle. It should name a layered system whose runtime can inspect the program it is asked to manage. This paper argued that the missing object is a formal representation of agents: typed terms for executable structure, inferred effects for authority, and cost grades for scheduling commitments before execution. The prototype is not a finished AgentOS, but it shows that this contract can be made concrete. Without it, “OS” remains a metaphor; with it, scheduling, protection, and optimization become system properties.

Acknowledgments

This work was supported by the “111 Center” (No. B26023) and the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (No. JYB2025XDXM118).

References

- [1] Kai Mei, Xi Zhu, Wujiang Xu, Wenyue Hua, Mingyu Jin, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. 2025. AIOS: LLM Agent Operating System. In *Conference on Language Modeling (COLM)*. arXiv:2403.16971.
- [2] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560* (2023).
- [3] Matthias Felleisen and Daniel P. Friedman. 1987. Control Operators, the SECD-Machine, and the λ -Calculus. In *Formal Description of Programming Concepts III*. North-Holland, 193–222.
- [4] C. L. Liu and James W. Layland. 1973. Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment. *J. ACM* 20, 1 (1973), 46–61. doi:10.1145/321738.321743
- [5] Dominic Orchard, Vilem-Benjamin Liepelt, and Harley Eades III. 2019. Quantitative Program Reasoning with Graded Modal Types. *Proceedings of the ACM on Programming Languages* 3, ICFP (2019), 110:1–110:30. doi:10.1145/3341714
- [6] Tomas Petricek, Dominic Orchard, and Alan Mycroft. 2014. Coeffects: A Calculus of Context-Dependent Computation. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 123–135. doi:10.1145/2628136.2628160
- [7] Jan Hoffmann, Klaus Aehlig, and Martin Hofmann. 2011. Multivariate Amortized Resource Analysis. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 357–370. doi:10.1145/1926385.1926427
- [8] Gordon D. Plotkin and Matija Pretnar. 2013. Handling Algebraic Effects. *Logical Methods in Computer Science* 9, 4:23 (2013), 1–36. doi:10.2168/LMCS-9(4:23)2013
- [9] Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. 1998. Language Primitives and Type Discipline for Structured Communication-Based Programming. In *European Symposium on Programming (ESOP) (LNCS, Vol. 1381)*. 122–138. doi:10.1007/BFb0053567